COMP502 ASSIGNMENT

Weighting: see Pass criteria on Moodle **Due date**: see Timetable on Moodle

Submission and Marking: see instructions on Moodle

Learning outcomes: Apply programming concepts and tools; Use Problem-solving and decision-making techniques which include logic and mathematical concepts, problem solving methods, critical thinking, abstract reasoning and systems.

Your code solution should follow the following guideline of Programming Best Practices:

1. Each program solution should have a program header at the beginning, written as paragraph comment statement. Program header should include author information (student name & ID), a pledge of honour, and a description that describes the purpose of the program. A sample program header is as follows:

```
'''
Author: John Smith

Pledge of Honour: I pledge by honour that this program is solely my own work.

Description: this program determines the state of iron based on its temperature.
'''
```

2. The following are a list of suggested **Programming Best Practices**:

- Variable names
  - start with a lowercase letter
  - adopt either one of the following styles but must maintain consistence:
    - camel case, i.e., capitalise first letter of each word
    - use underscore character to separate each word
  - should be concise, meaningful, i.e., descriptive of the data stored in the variable
- Program solution should not contain any unused variables, i.e., variables and any code that do not form part of program solution should be removed.
- Code readability: in-line space between characters and empty lines are used to increase code readability.
- Program solution should be internally documented. However, do not comment on EVERY line of code. Below is an example on generally how much inline documentation should be done:

```
# Create a variable and read from user for the purchase amount
purchase = float(input('Enter your purchase: '))
# Determines shipping charge based on purchase amount
if purchase > 0 and purchase < 50:
    shipping = 10
else:
    shipping = 0
# Calcualte total amount to pay
total = purchase + shipping
# Display total amount
print('Total to pay:', total)
```

TASKS TO COMPLETE

Note, a marking guide is attached at the end of this document.

Task 1: Introduction: simple drawing with turtle (reference materials on Moodle)

Using the turtle module, write a program to draw the first letter of your family name. The letter could be drawn using either straight lines or curves. For example, for the name Elton John, the letter J can be drawn as either one of the followings:



Task 2: Variables: arithmetic Calculation (reference materials on Moodle)

Write a program to ask from the user for the radius of a circle. The program then calculates and displays the circle perimeter and area.

Note, to calculate perimeter and area, you'll need the value of pi (symbol: π). Put the statement import math at the beginning of program to import the math module; then math.pi gives the π value. To calculate the square of the radius, use math.pow(radius, 2).
Formula: perimeter = radius * 2 * π

area = radius^2 * π

Sample run:

```
Enter radius: 2.5
Perimeter: 15.707963267948966
Area: 19.634954084936208
```

Task 3: F-strings: formatted output (reference materials on Moodle)

Workers are paid separately for their weekday hours and weekend hours. Write a program to ask the user for:

- The number of hours worked in weekdays
- The number of hours worked over weekends
- The weekday hourly pay rate (i.e., per hour)

Their weekend hourly rate is twice the weekday rate.

The program should calculate and display:

- The weekday total pay amount (calculated as weekday work hours multiplied by weekday hourly pay rate)
- The weekend total pay amount (calculated as weekend work hours multiplied by weekend hourly pay rate)
- The total pay amount (calculated as weekday total amount plus weekend total amount)

Note, the results must be:

- Rounded to 2 decimal points

- Displayed in two columns, with equal column width. See sample run output below.

Sample run:

```
Enter weekday hours: 10.5
Enter weekend hours: 5.5
Enter weekday hourly rate: 30

Payment details:

Weekday pay amount:     315.00
Weekend pay amount:     330.00
Total:                  645.00
```

Task 4: Selection: message of the day (reference materials on Moodle)

Write a program that gives you a different message for each day of the week. Ask for an integer from the user in the range 1 – 7, inclusive, where 1 – Monday, 2 - Tuesday, …, 7 - Sunday. As an example of running the program, if the input number is 5, the message can be as simple as "Good Friday". If the user inputs a number other than 1 - 7, the program should issue an appropriate message (e.g., mysterious day).

Sample run 1:

```
Enter a number (1-7): 6
Relaxing Saturday
```

Sample run 2:

```
Enter a number (1-7): 0
Mysterious day
```

Note:

- This program utilises multi-way selection (i.e., if-elif-else statement)
- Use "else" statement to error-trap input numbers other than 1 – 7.

Task 5: While loop: determine grocery discount eligibility (reference materials on Moodle)

Write a program that determines if a supermarket customer is eligible to receive discount.

1. The program should ask for the product price. Perform user input checking, i.e., the entered price must be greater than zero, or the user is prompted for re-entry until a valid input is received.
2. The program should then ask for the quantity. Perform user input checking, i.e., the entered quantity must greater than zero, or the user is prompted for re-entry until a valid input is received.
3. The program should then ask the user whether he/she has already claimed a discount during the month. Perform user input checking, i.e., the user response must either "y" or "n", or the user is prompted for re-entry until a valid response is received.

Hint: each of the above three steps would require a while-true loop.

4. The program should calculate the total shopping amount (i.e., multiply quantity by product price), then determine whether the customer is eligible for a discount. Both of the following two criteria must be met to be eligible:
   Criterion 1: The total shopping amount must be $500 or less, and
   Criterion 2: The customer hasn't claimed a discount.

Hint: this step would simply require an if-else statement.

Sample runs are as follows:

Sample run 1:
Enter product price (>0): -4
Invalid.
Enter product price (>0): 0
Invalid.
Enter product price (>0): 39.5

Enter quantity (>0): 0
Invalid.
Enter quantity (>0): -10
Invalid.
Enter quantity (>0): 8

Have you claimed the discount before (y/n): never
Invalid.
Have you claimed the discount before (y/n): not
Invalid.
Have you claimed the discount before (y/n): n

Total shopping amount: $316.0
Claimed discount: n
You are eligible

Sample run 2:
Enter product price (>0): $45.5
Enter quantity (>0): 10
Have you claimed the discount before (y/n): n

Total shopping amount: $455.0
Claimed discount: n
You are eligible

Sample run 3:
Enter product price (>0): $40.9
Enter quantity (>0): 7
Have you claimed the discount before (y/n): y

Total shopping amount: $286.3
Claimed discount: y
Sorry, you are not eligible

Task 6: For loop: draw a pattern of circles (reference materials on Moodle)

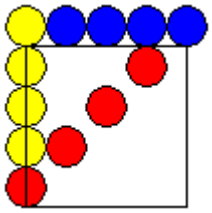
The program first asks the user for an integer number (between 2 - 6 inclusive). Perform user input checking, i.e., keep prompting for re-entry until a number in the range 2 – 6 is entered. The program then uses a loop to draw a triangle of circles shown below. The pattern consists four horizontal, four vertical, and four diagonal circles, with the right-angle at the top-left corner. You can choose your own colours for the drawing. A square is also drawn overlapping the bottom points of the circles at the four corners. The size of the square is calculated the number of circles (i.e., the number entered by the user) multiplied by the circle diameter (i.e., twice the radius).

The following sample run shows when the user enters the number 4. Your program must be general and works for any number in 2-6.

The circles used in the sample run output have radius of 10. The top-right corner of the square has (0, 0) as the coordinates.

Sample run:

```
Enter a number (2-6 incl.): 0
Invalid
Enter a number (2-6 incl.): 9
Invalid
Enter a number (2-6 incl.): 4
```



Task 7: List: display a list of records (reference materials on Moodle)

In the film industry, if a film's box office fails to break even, it's considered a box office bomb, thus losing money for the production company. The code template given below lists some films for which the box office is significantly lower than the budget. The records in the template have the format: `<film-id> <title> <budget> <box-office>`. Budget and box-office figures are in million dollars. Complete the program template by filling up the code block (annotated in the template), which displays the list of films in a tabular format as shown in the sample run below and the total number of films. Hint, use a for loop to iterate the list: `for rec in film_list`.

```
# Compose a list of film records.
film_list = []
film_list.append(['FM01', 'Stealth', 135, 80])
film_list.append(['FM02', 'Supernova', 90, 15])
film_list.append(['FM03', 'Robin Hood', 100, 85])
film_list.append(['FM04', 'Rollerball', 70, 26])
film_list.append(['FM05', 'Rust', 85, 20])

# Write code block here to display the film title with box-office, as well as the total
film count as per sample output.
....
```

Sample run:

```
Title               Box-office
------------------------------
Stealth             80
Supernova           15
Robin Hood          85
Rollerball          26
Rust                20

Film count: 5
```

Task 8: Function: process a list of records (reference materials on Moodle)

This task extends from the previous task. In the code template below, the program is broken into functions. The code for composing a list is turned into the function get_records, and the code that you'd completed in the previous task for displaying the list as a table should go in the function display_list.

- Write code for function search_film_by_id. The function asks the user to enter a film id. If there is a matching id in the film list, display the film title, and the budget loss. For example, the first film has 135 and 80 as the budget and box office respectively; the budget loss is then calculated as: loss = rec[2] – rec[3], where rec[2] is 135 and rec[3] is 80.
  If the entered id didn't match any of the films, display a no-match message, as the sample run output below.
- In the function main, functions get_records and display_list have been called for you. Complete the call to the function search_by_film_id.

```
def get_records():
    # Compose a list of film records.
    film_list = []
    film_list.append(['FM01', 'Stealth', 135, 80])
    film_list.append(['FM02', 'Supernova', 90, 15])
    film_list.append(['FM03', 'Robin Hood', 100, 85])
    film_list.append(['FM04', 'Rollerball', 70, 26])
    film_list.append(['FM05', 'Rust', 85, 20])
    return film_list

def display_list(data):
    # The code you'd completed in Task 7 goes here (with two additional columns)

def search_by_film_id(data):
    # write code for the function
    # Note, the function is none-value-return function

def main():
    rec_list = get_records()
    display_list(rec_list)

    #complete the call of search_by_film_id function here
    #Hint: the function is no-return function

main()
```

Sample run for display_list function:

```
ID        Title               Budget     Box-office
```

```
-------------------------------------------------
FM01      Stealth             135       80
FM02      Supernova           90        15
FM03      Robin Hood          100       85
FM04      Rollerball          70        26
FM05      Rust                85        20
```

Sample run scenario 1 for calling search_by_film_id function:

```
Enter film id: fm9
No matching film for fm9
```

Sample run scenario 2 for calling search_by_film_id function:

```
Enter film id: fm04
Film matching the given id: Rollerball
Budget loss for film FM04: 44
```

Task 9: Function: process a list of records (reference materials on Moodle)

This task extends from the previous task. In the code template below, complete the function get_total_budget_loss.

- Write code for function get_total_budget_loss. The function uses a loop to calculate the total budget of all films in the list. Note, this is **return** function, i.e., the calculated total is returned to the main function.
- Complete the function call in the main.
- A sample run output is given below.

Sample run for calling get_total_budget_loss function:

```
Total budget loss of all films: 254
```

```
def get_records():
    # code omitted (carried over from the previous task)

def get_total_budget_loss(data):
    # write code for the function
    # Note, the function is value-return function

def main():
    rec_list = get_records()

    #complete the call of get_total_budget_loss function here
    #Hint: the function is return function

main()
```

Task 10: File I/O: process supermarket sale records (reference materials on Moodle)

A supermarket merchandise has stores in many suburbs around the city. The following is a simplified log file containing store sales records. Each line in the file is comma separated and has the format:

```
<store_ID>,<employees>,<suburb_name>,<sale_volume>
```

Use the following content to create a plain text file, named *data.txt*. You should save the file in the same directory as your Python program to make easier accessing. Caution: do not leave empty lines at the end of the file.

```
Hm001,6,Frankton,42305.67
Hm002,10,Glenview,21922.22
Hm003,7,Silverdale,63277.9
Hm004,13,Glenview,83290.09
Hm005,21,Queenwood,81301.82
Hm006,14,Hillcrest,62333.3
Hm007,7,Frankton,28998.8
Hm008,19,Chartwell,51083.5
Hm009,6,Glenview,62155.72
Hm0010,8,Enderley,33075.1
Hm0011,10,Fairfield,61824.7
Hm0012,15,Rototuna,21804.8
Hm0013,11,Fairfield,62804.7
```

Implement the following functions:

- Write a value-return function **read_data(filename)**. The function takes a file name as parameter and reads the file content to create a list of records. Each record is represented as a list, elements of which correspond to a line of the file content. The function is a value-return function and returns the created list to its caller. Here is a sample line of code of how the function would be invoked in main(): `records=readdata('data.txt')`, where the variable `records` is the list of records from the file content. Note, the finally composed and returned list should have the elements converted to their intended data types (rather than all stored as strings), i.e., the list should have the following format:
  `[store_id_string, employees_int, suburb_name_string, sale_volume_float]`

- Write a none-value-return function **print_all_records(records)**. The function takes a list as parameter and displays the records, one record per line. The function should utilise formatting template to properly line up the record items in columns. Here is a sample code of how the function would be invoked in main(): `print_all_records(records)`, where records is the variable referring to the list created using the file content. Sample run output as follows:

```
     Store ID            Employees           Suburb              Sales
-------------------------------------------------------------------------------
1    Hm001               6                   Frankton            $42305.67
2    Hm002               10                  Glenview            $21922.22
3    Hm003               7                   Silverdale          $63277.90
4    Hm004               13                  Glenview            $83290.09
5    Hm005               21                  Queenwood           $81301.82
6    Hm006               14                  Hillcrest           $62333.30
7    Hm007               7                   Frankton            $28998.80
8    Hm008               19                  Chartwell           $51083.50
9    Hm009               6                   Glenview            $62155.72
10   Hm0010              8                   Enderley            $33075.10
11   Hm0011              10                  Fairfield           $61824.70
```

```
12    Hm0012              15                  Rototuna            $21804.80
13    Hm0013              11                  Fairfield           $62804.70
```

- Write a none-value-return function **write_data(filename, records)**. The function takes a file name and a list as parameters. The function should save the list of records to the specified file in the same format as in the file `data.txt`.

- Write a value_return function **get_min_emp_stores(records)**. The function takes a list of records as parameter, iterates using a loop, and calculates the minumum employee number of all stores. The calculated value is then used to filter and collect all stores that have the minimum number of employees. The collected records should be saved in a list and returned from the function. The returned record list is displayed to the screen in the main function.

  Sample run:

```
Stores with minimum number of employees:
     Store ID            Employees           Suburb              Sales
--------------------------------------------------------------------------------
1    Hm001               6                   Frankton            $42305.67
2    Hm009               6                   Glenview            $62155.72
```

- Write a value-return function **count_employees(records)**. The function takes a list as parameter. The function iterates through the record list and calculates the total number of employees from stores with sales volume more than $50,000. Note, this is a value-return function, i.e., the calculated count must be returned from the function.

  Note, the sample run below shows the result for the given text file. Your solution must be general and works for any record list of the same format.

  Sample run:

```
Total employees from stores with sales more than $50000: 101
```

- Write a none-value-return function **query_suburb_stats(records)**. The function is passed a list as parameter. The function begins by asking from the user for a suburb name. It then iterates through the list of records, calculates, and displays the following information about the user entered suburb:
  - The total sale volume of all stores in the suburb
  - The average sale volume per store

  The following is a sample case when user enters “frankton”. There are two stores in the Frankton suburb, thus,
  - total sale is calculated as $42305.67 + $28998.8 = $71304.47
  - average sale is calculated as $71304.47 / 2 = $35652.24

  Important note, the log file data.txt used in this program is sample only. Your solution, however, must be general and works with log files that possibly contain different data with more stores and/or suburbs.

  Sample run 1:

```
Enter branch name: hamilton east
No record was found for suburb Hamilton east
```

  Sample run 2:

```
Enter branch name: frankton
Total sale by frankton: $71304.47
Average sale by frankton: $35652.235
```

- Complete the main function below:

```
def main():

    data = read_data('data.txt')
    print_all_records(data)
    write_data('backup.txt', data)

    # complete the call to function get_min_emp_stores
    # note, it’s a value-return function

    # complete the call to function count_employees
    # note, it’s a value-return function

    # complete the call to function query_suburb_stats
```

Note, you are **NOT ALLOWED** to make changes to the header of aforementioned functions (including function name and parameters). All specified function parameters are to be utilised in the function body. Changing function header will result in the function being marked at maximum 50% of allocated marks.

Name: Score:

☐ Work submitted on Moodle

COMP502 ASSIGNMENT MARKING GUIDE

Total: 100

TASK 1 MARKING GUIDE	**Total: 3**
Letter drawn successfully, with proper presentation (e.g., not skewed or out of proportion)	2.5
Programming Best Practices	0.5
TASK 2 MARKING GUIDE	**Total: 7**
Correctly read user input, with appropriate data type	1.5
Calculate circle area and perimeter	4
Display calculated results	1
Programming Best Practices	0.5
TASK 3 MARKING GUIDE	**Total: 7**
Prompt the user for weekday hours, with clear prompt message	0.5
Prompt the user for weekend hours, with clear prompt message	0.5
Prompt the user for weekday hourly rate, with clear prompt message	0.5
Calculate and display weekday total, weekend total, and grand total	2.5
Format statements are used to round and column up output	2.5
Programming Best Practices	0.5
TASK 4 MARKING GUIDE	**Total: 8**
Prompt the user for a weekday number, with clear prompt message	1
Selection statement to show different messages for different weekdays	6
Display a message for any number outside range	0.5
Programming Best Practices	0.5
TASK 5 MARKING GUIDE	**Total: 10**
Read input for product price and perform input checking	1
Read input for quantity and perform input checking	1
Read input for whether the user has claimed discount before and perform input checking	1
Calculate total shopping amount	1
Display customer information (total shopping amount and discount claim)	1
Determine eligibility by applying criteria	4.5
Programming Best Practices	0.5
TASK 6 MARKING GUIDE	**Total: 10**
Read user input the number of circles to draw and perform input checking	2
Draw the circles according to user entered number	2
Draw the square with its size calculated using the user entered number multiplied by radius	2.5
Must utilise loops to draw the circles and square	2
Drawing is displayed in colour patterns	1
Programming Best Practices	0.5
TASK 7 MARKING GUIDE	**Total: 10**
Display table header	2
Display the list of records in tabular format using for loop and f-string	5.5
Display list count	2
Programming Best Practices	0.5

TASK 8 MARKING GUIDE	**Total: 10**
Complete code in function display_list	1.5
Function search_by_film_id	6
Complete function calls in main	2
Programming Best Practices	0.5
TASK 9 MARKING GUIDE	**Total: 10**
Complete code in function get_total_budget_loss	8
Complete the function call in main	1.5
Programming Best Practices	0.5
TASK 10 MARKING GUIDE	**Total: 25**
Function read_data()	3
Function print_all_records()	3
Function write_data()	2
Function get_min_emp_stores()	5
Function count_employees()	5
Function query_suburb_stats()	4
Complete function calls in main	2.5
Programming Best Practices	0.5